

Object-Oriented Programming with C++

ENSIA 2024–2025

Tutorial 12: Exception Handling

Exercise 1

The function below computes sales tax owed on a certain purchase. An error situation should occur whenever the user passes a negative price or negative sales tax into the function. In this exercise, you will modify the function such that the erroneous input is handled in three different ways.

```
1 double getSalesTax(double price, double taxRate = 0.625) {  
2     return price * taxRate;  
3 }
```

Tasks

1. Modify the function such that it outputs a special value in the case that one or both inputs are invalid.
2. Modify the function such that it prints out an error message and causes the program to immediately exit in the case that one or both of the inputs are invalid.
3. Modify the function such that it throws an object of type `InvalidInputException` in the case that either one or both inputs are invalid.

Exercise 2

A function that returns a special error code is often better implemented by throwing an exception instead. This way, the error code cannot be ignored or mistaken for valid data. The following class maintains an account balance.

```
1 class Account {  
2 private:  
3     double balance;  
4  
5 public:  
6     Account() { balance = 0; }  
7     Account(double initialDeposit) { balance = initialDeposit; }  
8  
9     double getBalance() { return balance; }  
10 }
```

```

11 // returns new balance or -1 if error
12 double deposit(double amount) {
13     if (amount > 0)
14         balance += amount;
15     else
16         return -1; // Code indicating error
17     return balance;
18 }
19
20 // returns new balance or -1 if invalid amount
21 double withdraw(double amount) {
22     if ((amount > balance) || (amount < 0))
23         return -1;
24     else
25         balance -= amount;
26     return balance;
27 }
28 };

```

Exercise 3

The Standard Template Library includes a class named `exception` that is the parent class for any exception thrown by an STL function. Therefore, any exception can be caught by this class. The following code sets up a try-catch block for STL exceptions:

```

1 #include <iostream>
2 #include <exception>
3 #include <string>
4 using namespace std;
5
6 int main() {
7     string s = "hello";
8     try {
9         cout << "No exception thrown." << endl;
10    }
11    catch (exception& e) {
12        cout << "Exception caught: " << e.what() << endl;
13    }
14    return 0;
15 }

```

Tasks

1. Modify the code so that an exception is thrown in the try block. You could try accessing an invalid index in a string using the `at` member function. What will be the output of your code?
2. Rewrite the same code using an array of `int` instead of a string.

Exercise 4

Define a stack class for storing a stack of elements of type `char`. A stack object should be of fixed size; the size is a parameter to the constructor that creates the stack object. When used in a program, an object of the stack class will throw exceptions in the following situations:

- Throw a `StackOverflowException` if the application program tries to push data onto a stack that is already full.
- Throw a `StackEmptyException` if the application program tries to pop data off an empty stack.

Defining the classes `StackOverflowException` and `StackEmptyException` is part of this program.

Tasks

1. Implement the class `stack`.
2. Write a suitable `main` program to test the exceptions.